Trace Equation Systems for Array Algorithms: Worked Examples

Notes on `examples/dafny/algorithms`

October 2026

These notes document examples of the folder `examples/dafny/algorithms` in the style of the paper *Trace Equation Systems for Program Verification*: for each program we show the trace equation system that Cuvée generates (`-teq`), the lemmas that the simplifier applies (`-simplify`), how they are instantiated, and the conceptual idea behind them. All lemmas are stated in the Dafny dialect of Cuvée at the beginning of the respective file; the lemma database `teq.dfy` is injected by `-teq`. We write $\lfloor X \rfloor$ for `butlast(X)`, the trace X without its last element, $\mathsf{current}(X)$ for its last element, and $\mathsf{counter}(i, c, n)$ for the counter $\langle i, i+c, \ldots, i+nc \rangle$ of n steps. The sections are ordered by the complexity of the proof, from a single loop with two lemmas to loops in sequence and nested loops with summaries and inductions; the last section documents an attempt that is open. Capital letters K, I, B, M denote traces, hatted letters $\hat{M}, \hat{J}, \hat{N}$ the fresh *sequences* that the summary of an inner loop introduces, one entry per iteration of the outer loop. $\mathsf{swap}(x, p, q)$ abbreviates the double store $x[p := x[q]][q := x[p]]$, which is how the swaps appear in the trace equations.

1 Linear Search: an Iteration Constraint Abstracted and Instantiated

1.1 The program

```
method lsearch(a: array<int, int>, n: int, x: int) returns (i: int)
  requires 0 <= n
  ensures 0 <= i <= n
  ensures i < n ==> a[i] == x
  ensures forall k: int :: 0 <= k < i ==> a[k] != x
{
  i := 0;
  while i < n && a[i] != x { i := i + 1; }
}
```

The first index of x among $a[0..n)$, or n. The loop test is data-dependent, so the number of iterations is not determined by n; the trace equation system records the test as the iteration constraint and its negation as the exit.

1.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad i = \mathsf{current}(I) \qquad I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ i, k \mapsto i + 1) \tag{1}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, i \mapsto i < n \wedge a[i] \neq x) \tag{2}$$

$$\neg(i < n \wedge a[i] \neq x) \tag{3}$$

$$\implies 0 \leq i \leq n \ \wedge\ (i < n \Rightarrow a[i] = x) \ \wedge\ (\forall k.\ 0 \leq k < i \Rightarrow a[k] \neq x)$$

With `-split`, the conclusion becomes four goals, one per conjunct, the last two with their premises as hypotheses: $i < n \Rightarrow a[i] = x$, and $0 \leq k < i \wedge a[k] = x \Rightarrow \mathsf{false}$. The simplifier's own rules do the following: I is a counter by `counter_def`, so $i = \mathsf{integer}(n')$; the constraint (2) is

distributed into $\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,n') \rfloor\ \mathsf{check}\ k \mapsto k < n)$, which invs_counter_upper resolves to $n' = 0 \vee \mathsf{integer}(n') \leq n$ (hence $0 \leq i \leq n$, the first two goals), and $\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,n') \rfloor\ \mathsf{check}\ k \mapsto a[k] \neq x)$, the search constraint proper; the exit (3) is $n \leq i \vee a[i] = x$, which with $i < n$ gives the third goal.

1.3 The lemmas

The last goal needs the search constraint at a witness: for $0 \leq k < i$, the constraint says $a[k] \neq x$. The paper's lemma database has no instantiation of iteration constraints at positions (`TODO.md`: no instantiation at a witness); the two lemmas of `lsearch.dfy`, since merged into `teq.dfy`, abstract the constraint by an atom and instantiate the atom:

$$\mathsf{not_occurs_selects_def}: \quad \mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(i,1,n) \rfloor\ \mathsf{check}\ k \mapsto a[k] \neq x) \iff \neg\mathsf{occurs}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(i,1,n) \rfloor))$$

$$\mathsf{select_not_occurs_selects}: \quad \neg\mathsf{occurs}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(i,1,n) \rfloor)) \ \wedge\ i \leq k < i + \mathsf{integer}(n) \implies (a[k] = x) \iff \mathsf{false}$$

The first matches the constraint with $i := 0$, $n := n'$ (the counter K is followed in the dependency position) and replaces it by the atom "x does not occur among the values of a at the positions $0, \ldots, n' - 1$". The second is a rule on the atom $a[k] = x$ of the last goal: its side conditions are the fact just produced, matched with $i := 0$, $n := n'$, and the bounds $0 \leq k < \mathsf{integer}(n')$, decided from $0 \leq k < i$ and $i = \mathsf{integer}(n')$. The hypothesis $a[k] = x$ becomes false and the goal closes.

This is the pattern of not_occurs_prefix_def and earlier_not_occurs_prefix of `teq.dfy` for reads $\mathsf{earlier}(k, xs)$ of a sequence, here for reads of an array: an iteration constraint becomes a named fact about the prefix visited, and a second lemma uses the fact at any position in the prefix. `TODO.md` notes that a side condition that is the iteration constraint itself would make the rule cycle (finding 19 of `NOTES.md`), which is why the abstraction by an atom is not merely convenient.

The file's header prescribes `-teq -split -simplify`, since without the split the disjunction of the two exits, $n \leq i \vee a[i] = x$, had to be split by the simplifier against the postcondition; with the current version the goal closes without `-split` as well.

2 Nested Sum: the Fragment Lemmas Alone

2.1 The program

```
method main(m: int, n: int) returns (c: int)
  requires 0 <= m && 0 <= n
  ensures c == m * n
{
  var i: int; var j: int;
  c := 0; i := 0;
  while i < m {
    j := 0;
    while j < n { j := j + 1; }
    c := c + j;
    i := i + 1;
  }
}
```

The smallest nested loop whose result depends on the inner loop: c sums the final values of j, which are all n. There is nothing to summarize but the inner counter, so this example introduces the *fragment lemmas* of `fragments.dfy`, which every nested example uses (Section 3.3 states the first two in full), and shows how an iteration constraint reaches a lemma of `teq.dfy`.

2.2 The trace equation system

$$K = \mathsf{counter}(0,1,n') \qquad I = \mathsf{counter}(0,1,n') \qquad \mathsf{integer}(n') = m \qquad c = \mathsf{sum}(\hat{J}) \tag{4}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{N}, \hat{K}'\ \mathsf{check}\ k, i, n'', K' \mapsto K' = \mathsf{counter}(0,1,n'')) \tag{5}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{K}', \hat{J}'\ \mathsf{check}\ k, i, K', J \mapsto J = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \tag{6}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}, \hat{J}'\ \mathsf{check}\ k, i, j, J \mapsto j = \mathsf{current}(J)) \tag{7}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}'\ \mathsf{check}\ k, i, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j < n)) \tag{8}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}\ \mathsf{check}\ k, i, j \mapsto n \le j) \tag{9}$$

$$\implies c = m \cdot n$$

The trace of c is $\mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \hat{J}\ \mathsf{do}\ c, k, j \mapsto c + j)$, which the introduction rule of sums in `teq.dfy` folds and the acceleration rule $\mathsf{current}(\mathsf{sums}(x, V)) = x + \mathsf{sum}(V)$ turns into $c = \mathsf{sum}(\hat{J})$: the sum of the final values of the inner loop. The task is to show that $\hat{J}$ consists of n repeated m times.

2.3 The lemmas and their instantiation

Two lemmas of `fragments.dfy`, stated in full in Section 3.3, apply with $X := \lambda k.\, 0$ and $c := 1$: invs_sums_counter (the trace of an inner variable increased by c over the inner counter K', which is $\mathsf{counter}(0,1,n'')$ in the same iteration, is a counter from its initial value $X(k)$) turns (6) into $\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(0,1,n''))$, and invs_current_counter (the final value of an inner counter) turns (7) into $\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n''))$. Two more fragment lemmas of `fragments.dfy` resolve the test and the exit; both are stated for an inner counter from 0, since their side conditions must be in the normal form of the facts, where a general initial value $X(k) + \mathsf{integer}(n'')$ would not match $\mathsf{integer}(n'')$ (a higher-order summand matches a single summand):

invs_invs_counter_upper_zero :

$$0 \le n \ \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(0,1,n''))$$
$$\implies$$
$$\mathsf{all}(\mathsf{for}\ ks, Js\ \mathsf{check}\ k, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ x \mapsto x < n))$$
$$\iff \mathsf{all}(\mathsf{for}\ ks, Ns\ \mathsf{check}\ k, n'' \mapsto \mathsf{integer}(n'') \le n)$$

invs_exit_counter_zero :

$$\mathsf{all}(\mathsf{for}\ ks, Ns, js\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n''))$$
$$\wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns\ \mathsf{check}\ k, n'' \mapsto \mathsf{integer}(n'') \le n) \implies$$
$$\mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto n \le j) \iff \mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto j = n)$$

The first matches the test (8), with the counter fragment as its side condition (which binds $Ns := \hat{N}$); the inner constraint $j < n$ over $\lfloor \mathsf{counter}(0,1,n'') \rfloor$ is the lemma invs_counter_upper of `teq.dfy`, $n'' = 0 \vee (0 < n \wedge \mathsf{integer}(n'') - 1 < n)$, which the nonnegative bound simplifies to $\mathsf{integer}(n'') \le n$. The second matches the exit (9), finds the two facts, and states the final value: $j = n$ in each outer iteration. It is an equivalence under its side conditions, since $j = \mathsf{integer}(n'') \le n$.

The goal is now $\mathsf{sum}(\hat{J}) = m \cdot n$ with the fact $\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{J}\ \mathsf{check}\ k, j \mapsto j = n)$. In the body of that iteration constraint the simplifier reads j as $\mathsf{earlier}(\mathsf{natural}(k), \hat{J})$, so the fact is

$$\mathsf{all}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,n') \rfloor\ \mathsf{check}\ k \mapsto \mathsf{earlier}(\mathsf{natural}(k), \hat{J}) = n)$$

(the counter K is followed in the dependency position), which consts_prefix_def of `teq.dfy` abstracts by the atom $\mathsf{consts_prefix}(n', n, \hat{J})$, and sum_consts_prefix with that atom as its side condition rewrites $\mathsf{sum}(\hat{J})$ to $n \cdot \mathsf{integer}(n')$. (Until October 10 this was sum_invs_const, with the iteration constraint itself as the side condition, which cycles in general, see `TODO.md`.) The goal $\mathsf{integer}(n') \cdot n = m \cdot n$ with $\mathsf{integer}(n') = m$ is closed by a lemma mul_eq_cancel, $(a \cdot n = b \cdot n) \Leftrightarrow (n = 0 \vee a = b)$, since the simplifier substitutes the solution of $\mathsf{integer}(n') = m$ into polynomials but not into the product.

2.4 The idea

The fragments of an inner loop are a trace equation system over bound variables. The simplifier would resolve them if the bound variables of one fragment saw the equations of the others, which they do not (`NOTES.md`, the two problems of nested loops). The fragment lemmas are the four steps of that resolution as rewrite rules: the inner trace is a counter, its current value is its length, the test bounds the length, the exit fixes it. The same four lemmas serve every example with an inner counter from 0 (`nested_mult`, `row_sums`, `transpose`); with another initial value the test and the exit are instead kept as side conditions of a summary (Section 3.4).

3 Selection Sort: Fragments, a Summary, and an Induction

3.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var i: int; var j: int; var m: int; var t: int;
  b := a; i := 0;
  while i < n {
    m := i; j := i + 1;
    while j < n {
      if b[j] < b[m] { m := j; }
      j := j + 1;
    }
    t := b[i]; b[i] := b[m]; b[m] := t;
    i := i + 1;
  }
}
```

Arrays are total maps $\mathsf{array}\langle\mathsf{int},\mathsf{int}\rangle$; $\mathsf{sorted}(a, lo, hi)$ states that $a[lo..hi)$ is nondecreasing, and $\mathsf{perm}(a, b)$ that a is a permutation of b as a total map, i.e. b composed with a bijection of the indices. Both are abstract predicates, declared without a definition; their properties are the lemmas below. Whole-array permutation needs no side conditions on the ranges of swapped positions: a swap at any two positions preserves it.

3.2 The trace equation system

Symbolic execution of the outer loop (Definition 4.4 of the paper) introduces the synthetic counter $K = \mathsf{counter}(0, 1, n')$ of n' iterations and one trace per modified variable. The inner loop is summarized during the execution of the body (Theorem 4.8): its own trace equation system is in terms of fresh variables, which become fresh *sequences* over the outer iterations, and it is recorded as the iteration constraint of the outer loop. After `-simplify` distributes the constraint over its conjuncts,

the goal is (variables renamed, the fragments of the inner loop grouped):

$$K = \mathsf{counter}(0,1,n') \qquad I = \mathsf{counter}(0,1,n') \qquad \mathsf{integer}(n') = n \qquad b = \mathsf{current}(B) \tag{10}$$

$$B = \mathsf{trace}(a,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{M}\ \mathsf{do}\ x,k,i,m \mapsto \mathsf{swap}(x,k,m)) \tag{11}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{M}, \hat{M}'\ \mathsf{check}\ k,i,m,M \mapsto m = \mathsf{current}(M)) \tag{12}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \lfloor B \rfloor, \hat{M}', \hat{J}'\ \mathsf{check}\ k,i,x,M,J \mapsto \\ M = \mathsf{trace}(k,\ \mathsf{for}\ \lfloor J \rfloor\ \mathsf{do}\ p,j \mapsto \mathsf{case}\ x[j] < x[p] \Rightarrow j\ \ \mathsf{case}\ x[p] \le x[j] \Rightarrow p)) \tag{13}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{N}, \hat{K}'\ \mathsf{check}\ k,i,n'',K' \mapsto K' = \mathsf{counter}(0,1,n'')) \tag{14}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{K}', \hat{J}'\ \mathsf{check}\ k,i,K',J \mapsto J = \mathsf{sums}(k+1, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \tag{15}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}', \hat{J}\ \mathsf{check}\ k,i,J,j \mapsto j = \mathsf{current}(J)) \tag{16}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}'\ \mathsf{check}\ k,i,J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j < n)) \tag{17}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}\ \mathsf{check}\ k,i,j \mapsto n \le j) \tag{18}$$

$$\implies \mathsf{sorted}(b,0,n) \wedge \mathsf{perm}(b,a)$$

Equation (11) is the outer array trace: in iteration k (with $i = k$, since I and K are equal counters that the simplifier does not identify), the positions k and m are swapped, where m is the result of the inner loop, the entry of the sequence $\hat{M}$ at k. The fragments (12)–(18) describe that inner loop: its result m is the current value of a *selection trace* M (12), which starts at k and moves to j whenever $x[j] < x[p]$, over the inner counter J (13); J is the trace of j, increased by 1 from $k+1$ over the synthetic inner counter K' of n'' steps (14), (15), with the final value j (16); the test $j < n$ holds in every inner iteration (17) and fails at the exit (18). The generic fold of `teq.dfy` already turned the trace of j into $\mathsf{sums}(k+1, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))$, since inside a fragment the equation $K' = \mathsf{counter}(0,1,n'')$ of the sibling fragment (14) is not followed (problem 1 of nested loops in `NOTES.md`).

3.3 Step 1: the inner counter (generic fragment lemmas)

The two lemmas of `fragments.dfy` relate fragments of one inner loop to each other. They are rewrite rules whose left-hand side is a fragment and whose side condition is a sibling fragment, matched against the facts of the goal, which binds the sequences that the left-hand side does not mention.

$$\begin{aligned}
&\mathsf{invs_sums_counter}: \\
&\quad \mathsf{all}(\mathsf{for}\ ks, Ks, Ns\ \mathsf{check}\ k,K,n'' \mapsto K = \mathsf{counter}(0,1,n'')) \implies \\
&\mathsf{all}(\mathsf{for}\ ks, Ks, Js\ \mathsf{check}\ k,K,J \mapsto J = \mathsf{sums}(X(k), \mathsf{map}(\mathsf{for}\ \lfloor K \rfloor\ \mathsf{yield}\ c))) \\
&\quad \iff \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k,n'',J \mapsto J = \mathsf{counter}(X(k),c,n''))
\end{aligned}$$

$$\begin{aligned}
&\mathsf{invs_current_counter}: \\
&\quad \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k,n'',J \mapsto J = \mathsf{counter}(X(k),c,n'')) \implies \\
&\mathsf{all}(\mathsf{for}\ ks, Js, js\ \mathsf{check}\ k,J,j \mapsto j = \mathsf{current}(J)) \\
&\quad \iff \mathsf{all}(\mathsf{for}\ ks, Ns, js\ \mathsf{check}\ k,n'',j \mapsto j = X(k) + \mathsf{integer}(n'') \cdot c)
\end{aligned}$$

The initial value $X(k)$ is a higher-order pattern variable (Miller fragment): it matches 0, k, $k+1$, or the value of a further dependency, so one lemma covers all inner counters. The first lemma matches fragment (15) with $ks := \lfloor K \rfloor$, $Ks := \hat{K}'$, $Js := \hat{J}'$, $X := \lambda k.\, k+1$, $c := 1$; its side condition is found as fragment (14), which binds $Ns := \hat{N}$. The unused dependency $\lfloor I \rfloor$ of the fragment is selected by no pattern dependency and dropped. The result replaces (15):

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k,n'',J \mapsto J = \mathsf{counter}(k+1,1,n'')). \tag{19}$$

The second lemma matches (16) with $Js := \hat{J}'$, $js := \hat{J}$, finds (19) as its side condition, and yields

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \hat{N}, \hat{J}\ \mathsf{check}\ k,n'',j \mapsto j = \mathsf{integer}(n'') + k + 1). \tag{20}$$

Conceptually, the fragment lemmas do by rewriting what the simplifier would do if the fragments of one inner loop shared their facts (the sharing was implemented and reverted, since it made rewriting cycle): they propagate the definition of a bound variable from one fragment into another. They are candidates for `teq.dfy`; they close the two minimal nested-loop problems `invs_counter_equation.dfy` and `invs_fragments.dfy`, and `nested_sum.dfy`, `nested_mult.dfy`, `distinct.dfy` and the sorting algorithms use them.

3.4 Step 2: the summary of the inner loop

The inner loop finds the position of the minimum of $x[k..n)$. We name that position by an abstract function $\mathsf{argmin}(x, lo, hi)$, declared without a definition, and state the summary as a rewrite rule whose side conditions are the fragments, in the forms they have after Step 1:

$$
\begin{aligned}
&\mathsf{current_selection_swaps}: \\
&\quad \mathsf{all}(\mathsf{for}\ ks, ms, Ms\ \mathsf{check}\ k, m, M \mapsto m = \mathsf{current}(M)) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, \lfloor \mathsf{trace}(a,\ \mathsf{for}\ ks, is, ms\ \mathsf{do}\ x, k, i, m \mapsto \mathsf{swap}(x, k, m))\rfloor, Ms, Js \\
&\qquad\quad \mathsf{check}\ k, x, M, J \mapsto \\
&\qquad\qquad M = \mathsf{trace}(k,\ \mathsf{for}\ \lfloor J\rfloor\ \mathsf{do}\ p, j \mapsto \mathsf{case}\ x[j] < x[p] \Rightarrow j\ \ \mathsf{case}\ x[p] \le x[j] \Rightarrow p)) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns, Js\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(k+1, 1, n'')) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ns, js\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n'') + k + 1) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto n \le j) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Js_2\ \mathsf{check}\ k, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J\rfloor\ \mathsf{check}\ j \mapsto j < n_2))\ \wedge\ Js_2 = Js\ \wedge\ n_2 = n \\
&\quad \Longrightarrow\ \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks, is, ms\ \mathsf{do}\ x, k, i, m \mapsto \mathsf{swap}(x, k, m))) \\
&\qquad\quad = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ x, k \mapsto \mathsf{swap}(x, k, \mathsf{argmin}(x, k, n))))
\end{aligned}
$$

The left-hand side is matched against $b = \mathsf{current}(B)$ with the definition (11) of B, which binds a, $ks := \lfloor K\rfloor$, $is := \lfloor I\rfloor$, $ms := \hat{M}$. The side conditions are then searched among the facts in order, each binding a new sequence: (12) binds $Ms := \hat{M}'$; (13) binds $Js := \hat{J}'$, where the dependency $\lfloor \mathsf{trace}(a,\ \mathsf{for}\ \ldots\ \mathsf{do}\ \rfloor)$ of the pattern is matched against $\lfloor B\rfloor$ through the definition of B; (19) binds $Ns := \hat{N}$; (20) binds $js := \hat{J}$; (18) binds n. The remaining condition is the inner test (17), which is matched by a duplicate Js_2 with a fresh bound n_2, identified afterwards by the equations $Js_2 = Js$ and $n_2 = n$: a side condition whose sequence variables are all bound is *decided* by the simplifier rather than matched against the facts, and the decision fails, since the facts keep $\lfloor I\rfloor$ as a dependency and rename the bound variables. The equations are closed and trivially decided.

Soundness of the lemma: given the side conditions, in iteration k the selection trace M scans the positions $k+1, \ldots, k+n''$ with $n'' + k + 1 \ge n$ (exit) and $k + 1 + n'' - 1 < n$ if $n'' > 0$ (test), hence exactly $k+1, \ldots, n-1$, so $\mathsf{current}(M)$ is a position of the minimum of $x[k..n)$, i.e. $\mathsf{argmin}(x, k, n)$ for any choice of argmin among the minima. The result is the trace of swaps of k with $\mathsf{argmin}(x, k, n)$, the *summary* of the whole algorithm as a trace of an abstract step. After the rewrite, $b = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor K\rfloor\ \mathsf{do}\ x, k \mapsto \mathsf{swap}(x, k, \mathsf{argmin}(x, k, n))))$.

Two workarounds are visible: the inner test is kept in its raw form (17) as a side condition instead of being resolved to a bound on n'', since the resolved form $k + 1 + \mathsf{integer}(n'') \le n$ is matched by `above_iterations_def` of `teq.dfy` (pattern $k < t$ with the dependency t synthesized as a map), which hides the fact; and the initial value $k+1$ and the final value $\mathsf{integer}(n'') + k + 1$ are written literally in the normal form of the simplifier, since a higher-order summand $X(k)$ matches a single summand only.

3.5 Step 3: sortedness by induction, permutation by a generic lemma

The postcondition $\mathsf{sorted}(b, 0, n)$ is now

$$\mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor K\rfloor\ \mathsf{do}\ x, k \mapsto \mathsf{swap}(x, k, \mathsf{argmin}(x, k, n)))), 0, n).$$

The invariant of selection sort after k iterations is that the prefix $x[0..k)$ is sorted and all of it is at most all of $x[k..n)$, written with a second abstract predicate $\mathsf{partitioned}(x, lo, mid, hi)$ (all of

$x[lo..mid)$ are at most all of $x[mid..hi)$). The induction is a lemma over a trace of arbitrary steps F:

$$\begin{aligned}&\mathsf{sorted_prefix_steps}:\\ &\quad n = \mathsf{integer}(m) \ \wedge\ \big(\forall x, k.\ 0 \le k < n \wedge \mathsf{sorted}(x,0,k) \wedge \mathsf{partitioned}(x,0,k,n)\\ &\qquad \Longrightarrow \mathsf{sorted}(F(x,k),0,k+1) \wedge \mathsf{partitioned}(F(x,k),0,k+1,n)\big)\\ &\quad \Longrightarrow\ \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,m)\rfloor\ \mathsf{do}\ x,k \mapsto F(x,k))),0,n) \iff \mathsf{true}\end{aligned}$$

The left-hand side matches with $F := \lambda x\, k.\ \mathsf{swap}(x,k,\mathsf{argmin}(x,k,n))$ and $m := n'$, through the definition $K = \mathsf{counter}(0,1,n')$ in the dependency position. The condition $n = \mathsf{integer}(m)$ is the fact $\mathsf{integer}(n') = n$ of (10); it fixes the number of steps to exactly n, since further steps would be unconstrained by the second condition (a lemma with $n \le \mathsf{integer}(m)$ would be unsound). The quantified condition is decided by the simplifier: F is beta-reduced, and the two conjuncts of the conclusion are rules on atoms,

$$\begin{aligned}&\mathsf{sorted_swap_argmin}:\\ &\quad k < n \wedge \mathsf{sorted}(x,0,k) \wedge \mathsf{partitioned}(x,0,k,n)\\ &\qquad \Longrightarrow \mathsf{sorted}(\mathsf{swap}(x,k,\mathsf{argmin}(x,k,n)),0,k+1) \iff \mathsf{true},\\ &\mathsf{partitioned_swap_argmin}:\\ &\quad k < n \wedge \mathsf{sorted}(x,0,k) \wedge \mathsf{partitioned}(x,0,k,n)\\ &\qquad \Longrightarrow \mathsf{partitioned}(\mathsf{swap}(x,k,\mathsf{argmin}(x,k,n)),0,k+1,n) \iff \mathsf{true},\end{aligned}$$

whose side conditions are the hypotheses of the quantified implication. The base case is inside the induction lemma: $\mathsf{sorted}(a,0,0)$ and $\mathsf{partitioned}(a,0,0,n)$ hold trivially (two further lemmas, sorted_empty and partitioned_empty, state them for completeness). This is the method of the paper's current_trace_bounded, an invariant scheme as a lemma with a quantified side condition: the lemma database proves the induction step, the shape of the invariant is the lemma's knowledge of selection sort, and no invariant inference takes place.

The postcondition $\mathsf{perm}(b,a)$ follows from a generic lemma on traces of swaps at positions that may depend on the state and the iteration (a higher-order pattern):

$$\begin{aligned}&\mathsf{perm_swaps_one}:\\ &\quad \mathsf{perm}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ x,k \mapsto \mathsf{swap}(x,P(x,k),Q(x,k)))),c)\\ &\quad \iff \mathsf{perm}(a,c)\end{aligned}$$

with $P := \lambda x\, k.\ k$ and $Q := \lambda x\, k.\ \mathsf{argmin}(x,k,n)$, and then perm_refl. The whole goal closes with `./Cuvee.sh -teq -simplify`; a wrong postcondition, e.g. $\mathsf{sorted}(b,0,n+1)$, leaves a goal.

3.6 What is generic and what is specific

Of the lemmas used, invs_sums_counter, invs_current_counter and perm_swaps_one are independent of the algorithm; sorted_prefix_steps is the induction for every algorithm that grows a sorted prefix (selection sort, and insertion sort in a variant with two dependencies); the summary current_selection_swaps and the two step lemmas are the algorithm's knowledge: what one inner loop computes, and why one step keeps the invariant. The abstract predicates and argmin are axiomatized by these lemmas only; a definition in Lean, as for `teq.dfy`, would make the step lemmas and the summary provable.

4 Bubble Sort: an Invariant Indexed by the Iteration

4.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
```

```
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var i: int; var j: int; var t: int;
  b := a; i := n;
  while 1 < i {
    j := 0;
    while j + 1 < i {
      if b[j + 1] < b[j] { t := b[j]; b[j] := b[j + 1]; b[j + 1] := t; }
      j := j + 1;
    }
    i := i - 1;
  }
}
```

4.2 The trace equation system

The outer loop modifies only b, and only through the inner loop, so its trace is the chain of the inner results, which the fold results names:

$$K = \mathsf{counter}(0, 1, n') \qquad I = \mathsf{counter}(n, -1, n') \qquad b = \mathsf{current}(B) \tag{21}$$

$$B = \mathsf{results}(a, \lfloor K \rfloor, \hat{B}) \quad \text{where} \quad \mathsf{results}(x_0, ks, bs) = \mathsf{trace}(x_0,\ \mathsf{for}\ ks, bs\ \mathsf{do}\ x, k, b \mapsto b)$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{B}, \hat{B}'\ \mathsf{check}\ k, i, b, B' \mapsto b = \mathsf{current}(B')) \tag{22}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor B \rfloor, \lfloor I \rfloor, \hat{B}', \hat{J}'\ \mathsf{check}\ k, x, i, B', J \mapsto B' = \mathsf{bubble_pass}(x, \lfloor J \rfloor)) \tag{23}$$

$$\text{where} \quad \mathsf{bubble_pass}(b, js) = \mathsf{trace}(b,\ \mathsf{for}\ js\ \mathsf{do}\ x, j \mapsto \mathsf{case}\ x[j+1] < x[j] \Rightarrow \mathsf{swap}(x, j, j+1)\ \ \mathsf{case}\ x[j] \le x[j+1] \Rightarrow x)$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}\ \mathsf{check}\ k, i, j \mapsto n \le j + k + 1) \tag{24}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}'\ \mathsf{check}\ k, i, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j + k + 1 < n)) \tag{25}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{J}, \hat{J}'\ \mathsf{check}\ k, i, j, J \mapsto j = \mathsf{current}(J)) \tag{26}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \hat{K}', \hat{J}'\ \mathsf{check}\ k, i, K', J \mapsto J = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \tag{27}$$

$$\ldots \wedge\ n \le \mathsf{integer}(n') + 1\ \implies\ \mathsf{sorted}(b, 0, n) \wedge \mathsf{perm}(b, a)$$

The inner trace folds into $\mathsf{bubble_pass}(x, \lfloor J \rfloor)$, a pass over the positions of the inner counter J, by a `{:fold}` function of the file, like sums in `teq.dfy`. The test and the exit are stated with $i = n - k$ already substituted: the pass of iteration k runs over the positions $0, \ldots, n - k - 2$.

4.3 The summary

$$\begin{aligned}
&\mathsf{current_results_bubble}: \\
&\quad \mathsf{all}(\mathsf{for}\ ks, bs, Bs\ \mathsf{check}\ k, b, B' \mapsto b = \mathsf{current}(B')) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, \lfloor \mathsf{results}(a, ks, bs) \rfloor, Bs, Js\ \mathsf{check}\ k, x, B', J \mapsto B' = \mathsf{bubble_pass}(x, \lfloor J \rfloor)) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, js\ \mathsf{check}\ k, j \mapsto len \le j + k + 1) \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Js_2\ \mathsf{check}\ k, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J \rfloor\ \mathsf{check}\ j \mapsto j + k + 1 < len))\ \wedge\ Js_2 = Js \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, js_3, Js_3\ \mathsf{check}\ k, j, J \mapsto j = \mathsf{current}(J))\ \wedge\ js_3 = js\ \wedge\ Js_3 = Js \\
&\quad \wedge\ \mathsf{all}(\mathsf{for}\ ks, Ks, Js\ \mathsf{check}\ k, K', J \mapsto J = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K' \rfloor\ \mathsf{yield}\ 1))) \\
&\quad \implies \mathsf{current}(\mathsf{results}(a, ks, bs)) \\
&\qquad = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ b, k \mapsto \mathsf{current}(\mathsf{bubble_pass}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(len - k - 1)) \rfloor))))
\end{aligned}$$

This is the summary in its original form (October 9), before the fragment lemmas: the inner counter is matched as sums, and the test and exit bind len. The left-hand side $\mathsf{current}(\mathsf{results}(a, ks, bs))$ is matched against $b = \mathsf{current}(B)$ through (21), binding $ks := \lfloor K \rfloor$ and $bs := \hat{B}$; (22) binds $Bs := \hat{B}'$, (23) binds $Js := \hat{J}'$, (24) binds $js := \hat{J}$ and $len := n$; (25), (26), (27) are matched through duplicates and the equations identify them, as in Section 3.4. The result is the trace of passes, the k-th over

$\lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n-k-1)) \rfloor$. Note that the arithmetic $n-k-1$ on the bound variable k is admissible on the right-hand side of a rule; on a left-hand side it would not be matched, which is why the original version of this file had a function $\mathsf{passlen}(len, k)$ for it, which the induction below makes unnecessary.

4.4 The induction with a clamped invariant

After k passes the suffix $b[n-k..n)$ is sorted and all of $b[0..n-k)$ are at most all of it. Stated with sorted and partitioned as in Section 3.5, the invariant makes sense only for $k \le n$, but the number of outer iterations n' is known only as $n \le n'+1$ together with the iteration constraint $1 < i$, a disjunction $n' = 0 \lor (2 < n \land n' < n)$ that the simplifier does not split, so the induction cannot fix the number of steps to $n-1$ (as Section 3.5 fixes it to n). Instead the invariant is a predicate indexed by the iteration, $\mathsf{bubbled}(b, k, n)$, which means the sorted suffix of length k once $k \le n$ and the whole range sorted once $k \ge n$; then every step keeps it, also the empty passes beyond $n-1$:

bubbled_pass :

$$p = len - k - 1 \land \mathsf{bubbled}(b, k, len)$$
$$\implies \mathsf{bubbled}(\mathsf{current}(\mathsf{bubble_pass}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(p)) \rfloor)), k+1, len) \iff \mathsf{true}$$

sorted_bubbled_steps :

$$len \le \mathsf{integer}(m) + 1$$
$$\land \ \big(\forall b, k.\ 0 \le k < \mathsf{integer}(m) \land \mathsf{bubbled}(b, k, len) \implies \mathsf{bubbled}(F(b,k), k+1, len)\big)$$
$$\implies \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, m) \rfloor\ \mathsf{do}\ b, k \mapsto F(b,k))), 0, len) \iff \mathsf{true}$$

The induction matches with $F := \lambda b\, k.\ \mathsf{current}(\mathsf{bubble_pass}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n-k-1)) \rfloor))$, $m := n'$, $len := n$. Its quantified condition is decided by bubbled_pass, whose length p is a variable with the equation $p = len-k-1$ as a side condition, since a pattern $len-k-1$ with two variables is not matched; the equation is closed after p is bound to $n-k-1$ by the left-hand side and decided by arithmetic. The lemma is sound for any number of steps m with $len \le m+1$: the invariant at $k = 0$ holds trivially, and $\mathsf{bubbled}(b, m, len)$ with $len - m \le 1$ is $\mathsf{sorted}(b, 0, len)$. For perm, perm_steps (the quantified condition $\forall b, k.\ \mathsf{perm}(F(b,k), b)$, decided by perm_bubble_pass) and perm_refl.

4.5 The idea

Two things differ from selection sort. The inner loop is summarized to a function of the previous array that is itself a trace, $\mathsf{current}(\mathsf{bubble_pass}(\ldots))$, so the outer trace is a trace of traces; the lemma database needs no knowledge of what a pass does beyond the two facts bubbled_pass and perm_bubble_pass. And the invariant is written as one predicate over the iteration, which absorbs the case distinction on the number of passes that the goal does not provide. The pattern "invariant as a predicate of (b, k), step lemma, generic induction" is the same as current_trace_bounded of `teq.dfy`, with the induction hypothesis as an abstract atom.

5 Cocktail Sort: Two Inner Loops and a Closed Side Condition

5.1 The program

```
method main(a: array<int, int>, n: int) returns (b: array<int, int>)
  requires 0 <= n
  ensures sorted(b, 0, n)
  ensures perm(b, a)
{
  var lo: int;
  var hi: int;
  var j: int;
  var p: int;
  var t: int;
```

```
  b := a;
  lo := 0;
  hi := n;
  while lo + 1 < hi
  {
    j := lo;
    while j + 1 < hi
    {
      if b[j + 1] < b[j] {
        t := b[j];
        b[j] := b[j + 1];
        b[j + 1] := t;
      }
      j := j + 1;
    }
    hi := hi - 1;
    j := hi - 1;
    p := hi - 2;
    while lo < j
    {
      if b[j] < b[p] {
        t := b[j];
        b[j] := b[p];
        b[p] := t;
      }
      j := j - 1;
      p := p - 1;
    }
    lo := lo + 1;
  }
}
```

Each outer iteration runs a forward bubble pass over $[lo, hi)$, which moves the maximum to $hi-1$, and a backward pass over $(lo, hi-1]$, which moves the minimum to lo; the backward pass uses two pointers j and $p = j-1$, like `insertion_sort2.dfy`, so that the positions of its swaps are counters.

5.2 The trace equation system

The outer loop modifies b only through the two inner loops, so its trace is again the chain of the results, $B = \mathsf{results}(a, \lfloor K \rfloor, \hat{B}'')$, where the intermediate results $\hat{B}'$ of the forward passes are the initial values of the backward passes. With $lo = k$ and $hi = n - k$ substituted,

$$K = Lo = \mathsf{counter}(0, 1, n') \qquad Hi = \mathsf{counter}(n, -1, n') \qquad b = \mathsf{current}(B) \qquad lo = \mathsf{integer}(n') \tag{28}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor \text{ check } k, lo, hi \mapsto 2k + 1 < n) \tag{29}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}', \hat{B}'_t \text{ check } k, lo, hi, c, C \mapsto c = \mathsf{current}(C)) \quad \mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor B \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}'_t, \hat{J}'_t \text{ check } k, x, lo, hi, \tag{30}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \hat{N}, \hat{J}'_t \text{ check } k, nn, J \mapsto J = \mathsf{counter}(k, 1, nn)) \quad \mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{J}' \text{ check } k, lo, hi, j \mapsto n \le j + k + 1) \tag{31}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{B}'', \hat{B}''_t \text{ check } k, lo, hi, b, B' \mapsto b = \mathsf{current}(B')) \quad \mathsf{all}(\text{for } \ldots, \hat{B}', \hat{B}''_t, \hat{J}''_t, \hat{P}''_t \text{ check } \ldots, c, B', J, P \mapsto \tag{32}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \hat{N}', \hat{J}''_t \text{ check } k, nn, J \mapsto J = \mathsf{counter}(n - k - 2, -1, nn)) \quad \mathsf{all}(\text{for } \lfloor K \rfloor, \hat{N}', \hat{P}''_t \text{ check } k, nn, P \mapsto P = \mathsf{counter}(n \tag{33}$$

$$\mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{J}'' \text{ check } k, lo, hi, j \mapsto j \le k) \quad \mathsf{all}(\text{for } \lfloor K \rfloor, \lfloor Lo \rfloor, \lfloor Hi \rfloor, \hat{J}'', \hat{J}''_t \text{ check } k, lo, hi, j, J \mapsto j = \mathsf{current}(J) \tag{34}$$

$$\ldots \wedge\ n \le \mathsf{integer}(n') + lo + 1 \implies \mathsf{sorted}(b, 0, n) \wedge \mathsf{perm}(b, a)$$

with bubble_pass as in the bubble sort and bubble_pass_back(c, js, ps) the trace of conditional swaps $\mathsf{swap}(x, j, p)$ over the two counters. All facts of the outer iteration carry the counters Lo and Hi as dependencies, which no body uses, and the outer exit is $n \le \mathsf{integer}(n') + lo + 1$ with $lo = \mathsf{integer}(n')$ a separate fact.

5.3 The summary and the induction

The summary current_results_shake takes the fragments of both inner loops as side conditions, twenty-one of them, in the manner of Section 3.4: the results and traces of each pass, the counters (whose initial values $n-k-2$, $n-k-3$ are higher-order $X(k)$, $Y(k)$, pinned by closed all conditions $\mathsf{all}(\mathsf{for}\ ks\ \mathsf{check}\ k \mapsto X(k) = len - k - 2)$, since a sum of a rule variable and a bound variable is not matched), the tests and the exits; the result is

$$\mathsf{current}(\mathsf{results}(a, ks, bs)) = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ ks\ \mathsf{do}\ b, k \mapsto \mathsf{shake}(b, k, len)))$$

with $\mathsf{shake}(b, k, len)$ abstract: the pass of iteration k. The induction is that of the bubble sort with the invariant $\mathsf{shaken}(b, k, len)$, both ends of length k sorted and the middle between them, the whole range once $2k + 1 \geq len$, and shaken_shake as the step.

The goal stayed open for a day although every piece was in place, and the diagnosis is the point of this section. Truncated copies of the summary with its first k side conditions showed that it fires with $k \leq 20$ and not with the 21st, $\mathsf{all}(\mathsf{for}\ ks, js_4, Js_3\ \mathsf{check}\ k, j, J \mapsto j = \mathsf{current}(J))$, the final values of the backward counter. All its variables were bound by earlier conditions, so the condition was closed and *decided* by the simplifier instead of matched against the fact (34), and the simplifier does not find a closed all among the facts (these carry the unused dependencies $\lfloor Lo \rfloor$, $\lfloor Hi \rfloor$, and their bound variables are renamed). The same condition for the forward pass had fresh variables js_5, Js_5 with identifying equations, the pattern of Section 3.4, and succeeded. With fresh js_6, Js_6 and $js_6 = js_4 \wedge Js_6 = Js_3$ the summary fires and perm closes.

Then $\mathsf{sorted}(b, 0, n)$ remained: the induction's bound $len \leq 2\,\mathsf{integer}(m) + 1$ is not a difference constraint, and the simplifier does not derive it from the two facts $n \leq \mathsf{integer}(n') + lo + 1$ and $lo = \mathsf{integer}(n')$. The lemma therefore states the bound in the goal's form,

$$\begin{aligned}\mathsf{sorted_shaken_steps}: \quad & l = \mathsf{integer}(m)\ \wedge\ len \leq \mathsf{integer}(m) + l + 1 \\ \wedge\ & \big(\forall b, k.\ 0 \leq k < \mathsf{integer}(m) \wedge \mathsf{shaken}(b, k, len) \Longrightarrow \mathsf{shaken}(F(b, k), k + 1, len)\big) \\ \Longrightarrow\ & \mathsf{sorted}(\mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, m) \rfloor\ \mathsf{do}\ b, k \mapsto F(b, k))), 0, len)\ \Longleftrightarrow\ \mathsf{true}\end{aligned}$$

with l an open variable matched against the equation of lo, after which the bound is literally a fact. The counter $\lfloor \mathsf{counter}(0, 1, m) \rfloor$ as a dependency matches $\lfloor K \rfloor$ through $K = \mathsf{counter}(0, 1, n')$ in the context, as in the bubble sort. Two things that were suspected were not the cause: the extra dependencies $\lfloor Lo \rfloor$, $\lfloor Hi \rfloor$ of the facts (a diagnostic with two and with four dependencies matched both) and the name n of the rule variable, which the program shares (it was renamed to len anyway).

5.4 The idea

Nothing new is needed for two inner loops: each pass is summarized by its own fold, and the summary of the outer iteration takes both. The lesson is about side conditions: a condition whose variables are all bound is decided, not matched, so every all condition that should be read off a fact must bind a fresh sequence and be identified afterwards; and a closed arithmetic condition must be the fact as the goal has it, since the decision procedure combines difference constraints only.

6 Merge: Pinned Guards and Open Trace Variables

6.1 The program

```
method main(a: array<int, int>, m: int, b: array<int, int>, n: int) returns (c: array<int, int>)
  requires 0 <= m && 0 <= n
  requires sorted(a, 0, m) && sorted(b, 0, n)
  ensures sorted(c, 0, m + n)
  ensures forall x: int :: count(x, selects(c, butlast(counter(0, 1, natural(m + n)))))
    == count(x, selects(a, butlast(counter(0, 1, natural(m)))))
     + count(x, selects(b, butlast(counter(0, 1, natural(n)))))
{
```

```
  var i: int; var j: int; var k: int;
  i := 0; j := 0; k := 0;
  while k < m + n {
    if n <= j || (i < m && a[i] <= b[j]) { c := c[k := a[i]]; i := i + 1; }
    else { c := c[k := b[j]]; j := j + 1; }
    k := k + 1;
  }
}
```

The second postcondition states the permutation as a multiset: for each value, its number of occurrences among the first $m + n$ elements of c is the sum of its occurrences in a and b, in the vocabulary of `teq.dfy`.

6.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad c = \mathsf{current}(C) \qquad k = \mathsf{integer}(n') \qquad m + n \le k \tag{35}$$

$$C = \mathsf{trace}(c_0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor, \lfloor J \rfloor\ \mathsf{do}\ c, k, i, j \mapsto \mathsf{case}\ G \Rightarrow c[k := a[i]]\ \ \mathsf{case}\ \neg G \Rightarrow c[k := b[j]]) \tag{36}$$

$$I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor J \rfloor\ \mathsf{do}\ i, k, j \mapsto \mathsf{case}\ G \Rightarrow i + 1\ \ \mathsf{case}\ \neg G \Rightarrow i) \tag{37}$$

$$J = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ j, k, i \mapsto \mathsf{case}\ G \Rightarrow j\ \ \mathsf{case}\ \neg G \Rightarrow j + 1) \tag{38}$$

where G abbreviates the test $n \le j \vee (i < m \wedge a[i] \le b[j])$ of the program. The pointers are traces over each other, (37) and (38): I has $\lfloor J \rfloor$ among its dependencies and J has $\lfloor I \rfloor$. A single loop, no fragments; the whole difficulty is in matching this shape.

6.3 The lemmas

The merge is named by an abstract list $\mathsf{merged}(a, m, b, n)$, the values of the result on $0, \ldots, m+n-1$, with two facts: it is sorted, and its counts are those of both ranges. The lemma on the loop rewrites the values of the result:

$$\begin{aligned}
&\mathsf{selects_merge_loop}: \\
&\quad s = m + n \ \wedge\ m + n \le \mathsf{integer}(nn) \ \wedge\ 0 \le m \wedge 0 \le n \\
&\quad \wedge\ (\forall i, j.\ G(i, j) \Leftrightarrow (n \le j \vee (i < m \wedge a[i] \le b[j]))) \\
&\quad \wedge\ (\forall i, j.\ U(i, j) = a[i]) \ \wedge\ (\forall i, j.\ V(i, j) = b[j]) \\
&\quad \wedge\ Iv = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, js\ \mathsf{do}\ i, k, j \mapsto \mathsf{case}\ G(i, j) \Rightarrow i + 1\ \ \mathsf{case}\ \neg G(i, j) \Rightarrow i) \\
&\quad \wedge\ is = \lfloor Iv \rfloor \\
&\quad \wedge\ Jv = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, is\ \mathsf{do}\ j, k, i \mapsto \mathsf{case}\ G(i, j) \Rightarrow j\ \ \mathsf{case}\ \neg G(i, j) \Rightarrow j + 1) \\
&\quad \wedge\ js = \lfloor Jv \rfloor \\
&\quad \Longrightarrow\ \mathsf{selects}(\mathsf{current}(\mathsf{trace}(c_0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, is, js\ \mathsf{do}\ c, k, i, j \mapsto \\
&\qquad\qquad \mathsf{case}\ G(i, j) \Rightarrow c[k := U(i, j)]\ \ \mathsf{case}\ \neg G(i, j) \Rightarrow c[k := V(i, j)])), \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(s)) \rfloor) \\
&\qquad = \mathsf{merged}(a, m, b, n)
\end{aligned}$$

and $\mathsf{sorted_merge_loop}$ is the same with the conclusion $\mathsf{sorted}(\mathsf{current}(\ldots), 0, s) \Leftrightarrow \mathsf{true}$ and the two sorted premises; $\mathsf{count_merged}$ gives

$$\begin{aligned}
\mathsf{count}(x, \mathsf{merged}(a, m, b, n)) = \mathsf{count}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(m)) \rfloor)) \\
+ \mathsf{count}(x, \mathsf{selects}(b, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor)).
\end{aligned}$$

Three workarounds of the matcher are built into this lemma, each isolated by experiments (`NOTES.md`, findings 14, 16 and the section on the pin):

1. **The pin.** A literal disjunctive guard $n \le j \vee (\ldots)$ in a pattern is never matched. The guard is the higher-order variable $G(i, j)$, which matches any guard over the bound variables i, j, and the quantified side condition $\forall i, j.\ G(i, j) \Leftrightarrow (n \le j \vee \ldots)$ pins it to the intended one; the simplifier decides the pin by beta reduction. The stored values $U(i, j) = a[i]$ and $V(i, j) = b[j]$ are pinned likewise. (A higher-order variable for an array read $a[i]$ alone would be instantiated by the array itself, finding 10, hence the pin instead of a literal $a[i]$.)

2. **The open trace variable.** The lemma must know that is and js are the pointer traces, which the pattern cannot say, since each is defined over the other. A closed side condition $is = \lfloor \mathsf{trace}(0,\ \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots) \rfloor$ is decided by the simplifier and fails (it does not expand $\lfloor I \rfloor$). Instead $Iv = \mathsf{trace}(0,\ \mathsf{for}\ \ldots, js\ \mathsf{do}\ \ldots)$ with an *open* trace variable Iv is matched against the hypothesis $I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor J \rfloor\ \mathsf{do}\ \ldots)$ (37), which binds $Iv := I$ (and the counter $\lfloor \mathsf{counter}(0, 1, nn) \rfloor$ matches $\lfloor K \rfloor$ through its definition); then $is = \lfloor Iv \rfloor$ is closed and trivially true.
3. **The whole atom as the left-hand side.** A variable argument of an uninterpreted function is not expanded by its definition when the simplifier rewrites below it: $\mathsf{sorted}(c, 0, m + n)$ with $c = \mathsf{current}(C)$ is not matched by a left-hand side $\mathsf{current}(\mathsf{trace}(\ldots,\ \mathsf{for}\ \ldots\ \mathsf{do}\ ))$, but $\mathsf{sorted}(\mathsf{current}(\mathsf{trace}(\ldots,\ \mathsf{for}\ \ldots\ \mathsf{do}\ )), 0, s)$ as the whole atom is, and $\mathsf{selects}(\mathsf{current}(\ldots), \ldots)$ with $\mathsf{selects}$ as the head, inside count and the quantifier. The length is a variable s with $s = m + n$ as a side condition, since a pattern $m + n$ of two variables is not matched.

The remaining condition $m + n \leq \mathsf{integer}(nn)$ is the exit $m + n \leq k$ with $k = \mathsf{integer}(n')$ of (35): after at least $m+n$ iterations the positions $0, \ldots, m+n-1$ are those of the merge, whatever further iterations store beyond them, which makes the lemma sound without fixing the number of iterations exactly (the goal knows it only as a disjunction).

6.4 The idea

The loop computes a function of its inputs that has a name in the specification's vocabulary, merged, and the lemma database relates the loop to that name and the name to the properties. The matching problems of mutually dependent traces are solved by two general devices, the pin and the open trace variable, which apply to every literal pattern that the matcher does not handle: in `insertion_sort.dfy` with one pointer, the positions j and $j - 1$ of the swaps are pinned in the same way, which closed a file that had been open since the start.

7 Counting Sort: Loops in Sequence and Atoms for Hypotheses

7.1 The program

```
method main(a: array<int, int>, n: int, k: int) returns (b: array<int, int>)
  requires 0 <= n && 0 < k
  requires keys_below(a, n, k)              // forall i :: 0 <= i < n ==> 0 <= a[i] < k
  ensures sorted(b, 0, n)
  ensures forall x: int :: count(x, selects(b, butlast(counter(0, 1, natural(n)))))
    == count(x, selects(a, butlast(counter(0, 1, natural(n)))))
{
  var c: array<int, int>; var i: int; var v: int; var s: int; var t: int;
  v := 0;
  while v < k { c := c[v := 0]; v := v + 1; }
  i := 0;
  while i < n { c := c[a[i] := c[a[i]] + 1]; i := i + 1; }
  s := 0; v := 0;
  while v < k { t := c[v]; c := c[v := s]; s := s + t; v := v + 1; }
  i := 0;
  while i < n { b := b[c[a[i]] := a[i]]; c := c[a[i] := c[a[i]] + 1]; i := i + 1; }
}
```

Four loops in sequence: the histogram is zeroed, filled, replaced by its prefix sums (the start position of each value), and the elements are scattered to their start positions, which are incremented.

7.2 The trace equation system

Each loop contributes a trace whose current value is the initial value of the next; the simplifier has already applied the introduction rules of `teq.dfy` (stores, consts, sums, selects) when the goal is

printed (V, V_2, K_2 are counters of k' steps with $\mathsf{integer}(k') = k$, K, I, K_3, I_3 counters of n' steps with $\mathsf{integer}(n') = n$):

$$c_1 = \mathsf{current}(\mathsf{stores}(c_0, \lfloor V \rfloor, \lfloor \mathsf{consts}(k', 0) \rfloor)) \tag{39}$$

$$c_2 = \mathsf{current}(\mathsf{trace}(c_1,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{do}\ c, k, i \mapsto c[a[k] := c[a[k]] + 1])) \tag{40}$$

$$c_3 = \mathsf{current}(\mathsf{stores}(c_2, \lfloor V_2 \rfloor, \lfloor S \rfloor)) \qquad S = \mathsf{sums}(0, \mathsf{selects}(c_2, \lfloor K_2 \rfloor)) \tag{41}$$

$$b' = \mathsf{current}(\mathsf{stores}(b, \mathsf{map}(\mathsf{for}\ \lfloor K_3 \rfloor, \lfloor C \rfloor, \lfloor I_3 \rfloor\ \mathsf{yield}\ k, c, i \mapsto c[a[k]]), \mathsf{selects}(a, \lfloor I_3 \rfloor))) \tag{42}$$

$$C = \mathsf{trace}(c_3,\ \mathsf{for}\ \lfloor K_3 \rfloor, \lfloor I_3 \rfloor\ \mathsf{do}\ c, k, i \mapsto c[a[k] := c[a[k]] + 1]) \tag{43}$$

The scatter (42) is a stores whose keys are the positions read from the table C as it is incremented, and whose values are the elements of a.

7.3 The lemma chain

The histogram is characterized at each value (the generic lemma current_trace_add_at of `teq.dfy` is for a fixed key; here the key is $a[k]$):

$$\begin{aligned}&\mathsf{current_trace_histogram}:\\ &\quad \mathsf{current}(\mathsf{trace}(c_0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n) \rfloor, hs\ \mathsf{do}\ c, k, h \mapsto c[a[k] := c[a[k]] + 1]))[v]\\ &\qquad = c_0[v] + \mathsf{count}(v, \mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,n) \rfloor))\end{aligned}$$

It fires on the reads $c_2[v]$ inside $\mathsf{selects}(c_2, \lfloor K_2 \rfloor)$ of (41), with $c_1[v] = 0$ from (39) by current_stores of `teq.dfy`, so the prefix sums become

$$S = \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor K_2 \rfloor, \lfloor V_2 \rfloor\ \mathsf{yield}\ v, h \mapsto \mathsf{count}(v, \mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,n') \rfloor)))).$$

The table of start positions is then named by a lemma whose left-hand side is this whole term:

$$\begin{aligned}&\mathsf{rank_table_def}:\\ &\quad \mathsf{current}(\mathsf{stores}(c_0, \lfloor \mathsf{counter}(0,1,kk) \rfloor, \lfloor \mathsf{sums}(0, \mathsf{map}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,kk) \rfloor, hs\ \mathsf{yield}\ v, h \mapsto\\ &\qquad \mathsf{count}(v, \mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,n) \rfloor)))) \rfloor)) = \mathsf{rank_table}(c_0, a, n, kk)\end{aligned}$$

which rewrites c_3 to $\mathsf{rank_table}(c_2, a, n', k')$, the number of elements of $a[0..n)$ below each value $v < k$ (and c_2 beyond). Naming the table is essential: the scatter lemma below needs the bound kk of the keys, and a side condition $\mathsf{ranks}(c_0, a, n, kk)$ with kk open would be matched by no fact, while $\mathsf{rank_table}(c_2, a, n, kk)$ in the pattern binds kk. Finally the scatter:

$$\begin{aligned}&\mathsf{sorted_scatter}:\\ &\quad s = \mathsf{integer}(n)\ \wedge\ \mathsf{keys_below}(a, s, kb)\ \wedge\ kb = \mathsf{integer}(kk)\ \wedge\ Iv = \mathsf{counter}(0,1,n)\ \wedge\ hs_3 = \lfloor Iv \rfloor\\ &\quad \Longrightarrow\ \mathsf{sorted}(\mathsf{current}(\mathsf{stores}(b_0, \mathsf{map}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,n) \rfloor, \lfloor T \rfloor, hs_2\ \mathsf{yield}\ k, c, h \mapsto c[a[k]]),\\ &\qquad\qquad \mathsf{selects}(a, hs_3))), 0, s)\ \Longleftrightarrow\ \mathsf{true}\\ &\qquad \text{where } T = \mathsf{trace}(\mathsf{rank_table}(c_2, a, n, kk),\\ &\qquad\quad \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n) \rfloor, hs\ \mathsf{do}\ c, k, h \mapsto c[a[k] := c[a[k]] + 1])\end{aligned}$$

and count_scatter with the conclusion

$$\mathsf{count}(x, \mathsf{selects}(\mathsf{current}(\ldots), \lfloor \mathsf{counter}(0,1,n) \rfloor)) = \mathsf{count}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(0,1,n) \rfloor)).$$

The left-hand side binds $n := n'$, $kk := k'$, $hs_3 := \lfloor I_3 \rfloor$ and the rest; the counter of the values' selects is tied by the open trace variable Iv (Section 6.3), since $\lfloor I_3 \rfloor$ and $\lfloor \mathsf{counter}(0,1,n') \rfloor$ are not identified by a closed equation. The precondition on the keys is the atom keys_below of the specification with an open bound kb, matched against the fact and then identified with $\mathsf{integer}(kk)$: a quantified hypothesis $\forall i.\ 0 \le i < n \Rightarrow 0 \le a[i] < k$ is matched by no side condition, open or closed, so the specification uses the atom (as `count_occurrences.dfy` uses the vocabulary of `teq.dfy` in its postcondition).

Soundness of sorted_scatter: with the start positions correct for all keys, element $a[k]$ is stored at (number of elements below $a[k]$) + (number of equal elements before position k), which is a bijection of $0, \ldots, n-1$ that is monotone in the value, so the result is sorted on $0, \ldots, n-1$ and has the elements of a. Both are asserted, not proved; the lemma is the algorithm's correctness argument in one statement, over the exact shape of the two last loops.

7.4 The idea

Loops in sequence compose through the current values: each loop's trace has the previous result as its initial value, and the acceleration lemmas of `teq.dfy` (stores read at a position, sums, consts) push the characterization of one loop into the next without any lemma specific to counting sort, up to the table of start positions. The algorithm-specific knowledge is concentrated in naming the table and in the scatter lemma, whose left-hand side is the composition of the last two loops. What could not be expressed is a quantified precondition as a side condition; the specification's vocabulary must provide an atom for it.

8 First Duplicate: a Search with an Early Exit

8.1 The program

```
method main(a: array<int, int>, n: int) returns (r: bool)
  requires 0 <= n
  ensures r ==> !distincts(selects(a, butlast(counter(0, 1, natural(n)))))
  ensures !r ==> distincts(selects(a, butlast(counter(0, 1, natural(n)))))
{
  var i: int; var j: int;
  r := false; i := 0;
  while i < n && !r {
    j := i + 1;
    while j < n && !r {
      if a[i] == a[j] { r := true; }
      j := j + 1;
    }
    i := i + 1;
  }
}
```

The nested loop of `distinct.dfy` (which is verified) with both loops stopped by the flag: the search for a duplicate ends at the first one. The postcondition uses distincts of the lemma database, and is stated as two implications rather than the equivalence $r \Leftrightarrow \neg\mathsf{distincts}(\ldots)$, for a reason explained below.

8.2 The trace equation system

The outer loop's constraint includes the flag, $\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor R\rfloor, \lfloor I\rfloor\ \mathsf{check}\ k, r, i \mapsto \neg r)$, so r is false at the head of every outer iteration, and the simplifier uses this inside the inner trace:

$$K = I = \mathsf{counter}(0, 1, n') \qquad r = \mathsf{current}(R) \qquad R = \mathsf{results}(\mathsf{false}, \lfloor K\rfloor, \hat{R}) \qquad i = \mathsf{integer}(n') \tag{44}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor R\rfloor, \lfloor I\rfloor\ \mathsf{check}\ k, r, i \mapsto \neg r) \tag{45}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}, \hat{R}'\ \mathsf{check}\ k, i, r, R' \mapsto r = \mathsf{current}(R')) \tag{46}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}', \hat{J}'\ \mathsf{check}\ k, i, R', J \mapsto \quad R' = \mathsf{trace}(\mathsf{false},\ \mathsf{for}\ \lfloor J\rfloor\ \mathsf{do}\ r, j \mapsto \mathsf{case}\ a[k] = a[j] \Rightarrow \mathsf{true}\ \ \mathsf{case}\ a[k] \neq a[j] \Rightarrow \mathsf{false})) \tag{47}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}, \hat{J}'\ \mathsf{check}\ k, n'', J \mapsto J = \mathsf{counter}(k+1, 1, n'')) \tag{48}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}, \hat{J}\ \mathsf{check}\ k, n'', j \mapsto j = \mathsf{integer}(n'') + k + 1) \tag{49}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{J}'\ \mathsf{check}\ k, i, J \mapsto \mathsf{all}(\mathsf{for}\ \lfloor J\rfloor\ \mathsf{check}\ j \mapsto j < n)) \tag{50}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}'\ \mathsf{check}\ k, i, R' \mapsto \mathsf{all}(\mathsf{for}\ \lfloor R'\rfloor\ \mathsf{check}\ r \mapsto \neg r)) \tag{51}$$

$$\mathsf{all}(\mathsf{for}\ \lfloor K\rfloor, \lfloor I\rfloor, \hat{R}, \hat{J}\ \mathsf{check}\ k, i, r, j \mapsto \neg(j < n \wedge \neg r)) \tag{52}$$

$$\neg(i < n \wedge \neg r) \implies r \Leftrightarrow \neg\mathsf{distincts}(\mathsf{selects}(a, \lfloor\mathsf{counter}(0, 1, \mathsf{natural}(n))\rfloor))$$

(after the fragment lemmas of Section 3.3, which gave (48) and (49)). The inner update (47) does not mention the previous value of the flag, since the inner constraint $\neg r$ (51) replaced it by false: the inner trace is a sequence of comparisons with $a[k]$. The inner exit (52) is the negation of the test

$j < n \wedge \neg r$: either all later positions were visited, or the flag was set. The outer exit $\neg(i < n \wedge \neg r)$ is a disjunction, $n \leq i \vee r$, with one goal per postcondition.

8.3 The lemmas

The summary of the inner loop in the style of Section 3.4 is a rewrite of the fragment (46) into a fact about the results: with (47)–(52) as side conditions (duplicated and identified by equations where closed),

$$\mathsf{all}(\mathsf{for}\ ks, rs, Rs\ \mathsf{check}\ k, r, R' \mapsto r = \mathsf{current}(R')) \iff \mathsf{all}(\mathsf{for}\ ks, rs\ \mathsf{check}\ k, r \mapsto r = \neg\mathsf{nodup}(a, k, n))$$

where $\mathsf{nodup}(a, k, n)$ is the abstract "$a[k]$ does not occur among $a[k+1..n)$". This is sound in both exit cases of the inner loop: if the flag was set, a later duplicate was found; if j reached n, all later positions were compared. The outer trace is then $\mathsf{results}(\mathsf{false}, \lfloor K \rfloor, \hat{R})$ with $\hat{R}$ characterized pointwise, and the simplifier itself turns it into $\mathsf{results}(\mathsf{false}, \lfloor K \rfloor, \mathsf{map}(\mathsf{for}\ \lfloor K \rfloor\ \mathsf{yield}\ k \mapsto \neg\mathsf{nodup}(a, k, n)))$.

The outer loop is summarized on the atom $\mathsf{distincts}(\ldots)$ of the postcondition, by one rule per case of the exit. If the flag is set, it is the result of the last iteration, which found a duplicate:

$$\mathsf{distincts_flag_set}: \quad Kv = \mathsf{counter}(0, 1, m) \ \wedge\ flag = \mathsf{current}(\mathsf{results}(\mathsf{false}, \lfloor Kv \rfloor, \mathsf{map}(\mathsf{for}\ \lfloor Kv \rfloor\ \mathsf{yield}\ k \mapsto \neg\mathsf{nodup}(a, k, n))))$$
$$\wedge\ flag\ \wedge\ \mathsf{integer}(m) \leq n \implies \mathsf{distincts}(\mathsf{selects}(a, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor)) \iff \mathsf{false}$$

If all iterations ran, $n \leq \mathsf{integer}(m)$, and the flag was reset at the head of each, (45), then no element before the last has a later duplicate (the flag at the head of iteration k is the result of iteration $k-1$) and the last has none:

$$\mathsf{distincts_flags_reset}: \quad Kv = \mathsf{counter}(0, 1, m) \ \wedge\ Rv = \mathsf{results}(\mathsf{false}, \lfloor Kv \rfloor, rs)$$
$$\wedge\ \mathsf{all}(\mathsf{for}\ \lfloor Kv \rfloor, hs\ \mathsf{check}\ k, h \mapsto \neg h) \ \wedge\ hs = \lfloor Rv \rfloor \ \wedge\ \mathsf{all}(\mathsf{for}\ \lfloor Kv \rfloor, rs_2\ \mathsf{check}\ k, h \mapsto h = \neg\mathsf{nodup}(a, k, n)) \ \wedge\ rs_2 = rs \ \wedge$$
$$\implies \mathsf{distincts}(\mathsf{selects}(a, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n)) \rfloor)) \iff \mathsf{true}$$

Three details made these fire. The outer counter and the flag trace are the variables K and R of the goal with their equations (44) as hypotheses, so a rule with $\lfloor \mathsf{counter}(0, 1, m) \rfloor$ as an *argument* never matches; the open trace variables Kv, Rv are matched against the hypotheses (Section 6). The fact of the flags at the heads (45) carries the counter $\lfloor I \rfloor$ as an unused dependency, which the simplifier drops from a closed instance of the condition, after which it is not found among the facts; hence the open list hs, identified with $\lfloor Rv \rfloor$ afterwards. And the exit $n \leq i \vee r$ is needed as a closed condition: with the postcondition as an equivalence, the goal's conclusion is $i < n \wedge \neg r \ \vee\ r \Leftrightarrow \neg\mathsf{distincts}(\ldots)$, which `-split` does not split, and in the context of the second disjunct neither literal of the negated first is available, so the condition is not decided; probes (rules with the real left-hand side and the first k conditions) located it as the only failing one. As two implications, each goal has its literal of the exit, r respectively $n \leq i$ with $i = \mathsf{integer}(n')$, in the context of the atom.

An earlier set of lemmas closed both goals and also the negated postcondition: the rule for the first step had the summary fragment as an open side condition, and a rule $\mathsf{nodup}(a, k, n) \Leftrightarrow \mathsf{true}$ under the hypothesis $\mathsf{distincts}(\ldots)$ for the second. An all side condition is matched in the simplifier's context of the fact's combinator, where rules fire: under the hypothesis distincts, the rule rewrote the condition to (45), a fact, so distincts was derived from itself (finding 13 of `NOTES.md`). The rules above take the atom distincts as left-hand side and never rewrite nodup.

8.4 The idea

An early exit is a data-dependent number of iterations with a disjunctive exit; the summary of the inner loop must be stated so that it holds in both cases, which the abstract atom nodup affords, since it describes the result rather than the number of iterations; the outer loop is summarized on the atom of the postcondition with one rule per case of its exit, each with the case as a closed condition. The soundness lesson stands: a side condition that other rules can simplify under the conclusion to be proved is a circular justification, and the safe form of side conditions is an atom that is matched literally (as consts_prefix_def of the database does for a constraint, and as the summary itself does for the inner loop).

9 Binary Search: the Lower Bound (open)

9.1 The program

```
method main(a: array<int, int>, n: int, x: int) returns (lo: int)
  requires 0 <= n
  requires sorted(a, 0, n)
  ensures 0 <= lo <= n
  ensures forall k: int :: 0 <= k < lo ==> a[k] < x
  ensures forall k: int :: lo <= k < n ==> x <= a[k]
{
  var hi: int; var mid: int;
  lo := 0; hi := n;
  while lo < hi {
    mid := (lo + hi) / 2;
    if a[mid] < x { lo := mid + 1; } else { hi := mid; }
  }
}
```

The first position of $a[0..n)$ whose element is at least x, for a sorted array. The example `bsearch.dfy` of `examples/dafny/teq` verifies a binary search with the stronger precondition of partition points for all indices; here the precondition is $\mathsf{sorted}(a, 0, n)$, so the comparisons $a[m] < x$ are meaningful for $0 \le m < n$ only.

9.2 The trace equation system

$$K = \mathsf{counter}(0, 1, n') \qquad lo = \mathsf{current}(L) \qquad hi = \mathsf{current}(H) \qquad hi \le lo \tag{53}$$

$$L = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor H \rfloor\ \mathsf{do}\ lo, k, hi \mapsto \mathsf{case}\ G \Rightarrow (hi + lo)/2 + 1\ \ \mathsf{case}\ \neg G \Rightarrow lo) \tag{54}$$

$$H = \mathsf{trace}(n,\ \mathsf{for}\ \lfloor K \rfloor, \lfloor L \rfloor\ \mathsf{do}\ hi, k, lo \mapsto \mathsf{case}\ G \Rightarrow hi\ \ \mathsf{case}\ \neg G \Rightarrow (hi + lo)/2) \tag{55}$$

where G abbreviates $a[(hi + lo)/2] < x$

$$\mathsf{all}(\mathsf{for}\ \lfloor K \rfloor, \lfloor L \rfloor, \lfloor H \rfloor\ \mathsf{check}\ k, lo, hi \mapsto lo < hi) \tag{56}$$

As in the merge, the two variables are traces over each other, with a data-dependent guard that reads the array at a position computed from both.

9.3 The attempt

The intended lemmas name the result by an abstract function $\mathsf{lb}(a, n, x)$ with the facts $0 \le \mathsf{lb}(a, n, x) \le n$, $(a[k] < x) \Leftrightarrow \mathsf{true}$ for $0 \le k < \mathsf{lb}(a, n, x)$, and $(x \le a[k]) \Leftrightarrow \mathsf{true}$ for $\mathsf{lb}(a, n, x) \le k < n$ (all under $\mathsf{sorted}(a, 0, n)$), which close the postcondition once $lo = \mathsf{lb}(a, n, x)$ is known. The summary of the loop in the style of the merge (Section 6): the guard is the higher-order $G(lo, hi)$ pinned by $\forall lo, hi.\ G(lo, hi) \Leftrightarrow a[(hi + lo)/2] < x$, the trace of hi is tied by an open trace variable $Hv = \mathsf{trace}(n,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, \lfloor Lv \rfloor\ \mathsf{do}\ \ldots)$ with $hs = \lfloor Hv \rfloor$, and the left-hand side is $\mathsf{current}(\mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, hs\ \mathsf{do}\ lo, k, hi \mapsto \mathsf{case}\ G(lo, hi) \Rightarrow (hi + lo)/2 + 1\ \ \mathsf{case}\ \neg G(lo, hi) \Rightarrow lo))$, rewritten to $\mathsf{lb}(a, n, x)$. What the lemma must also know is that the loop has terminated, the exit $hi \le lo$ of (53): without it the current value of L is not the lower bound.

Stating the exit as a side condition $\mathsf{current}(Hv) \le \mathsf{current}(Lv)$ makes the run exceed the time limit: to decide the condition, the simplifier rewrites $\mathsf{current}(Lv)$, which is the left-hand side of the very lemma, whose conditions decide it again (finding 20 of `NOTES.md`). Stating it as a comparison $h \le l$ of two open variables, matched against the hypothesis $hi \le lo$, makes the lemma fire, and the four facts about lb then close the three postconditions: the goal is closed, and a wrong postcondition leaves a goal. But h and l are not tied to the traces, so the rule is not a sound lemma (in this goal its conclusion holds whichever comparison the condition matched, since the exit does hold in the final state). Every sound tie fails for a reason of the engine, located with probes: the instance of a side condition is simplified before it is decided or matched, so $l = \mathsf{current}(Lv)$ with Lv bound re-enters the lemma like the first form, whether closed or open. With Lv open, $l = \mathsf{current}(Lv)$

binds both from the hypothesis $lo = \mathsf{current}(L)$ without re-entrance; but then the left-hand side, matched against $\mathsf{current}(L)$ through the definition of the trace variable L, turns out to have left its dependency variables nn and hs unbound (a probe fired with "variables of a rule not instantiated"), and the trace condition that would bind them, $Lv = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, hs\ \mathsf{do}\ \ldots)$, no longer matches once Lv is bound, since its instance is simplified with L replaced by its definition.

9.4 The idea, and what is missing

Binary search is the paper's acknowledged weakness: the loop computes a bound by narrowing, and only a partial description of the traces (monotone lo, monotone hi, $lo \leq \mathsf{lb} \leq hi$ throughout) is needed, not their values. `bsearch.dfy` handles it by abstracting the comparisons with the array into comparisons of indices with partition points (lemmas $(a[m] < x) \Leftrightarrow m < p$ without side conditions), by the named monotone traces incs and decs whose bounds are their facts, and by a cut after the loop. With sorted instead of partition points, the comparison lemma needs $0 \leq m < n$, which in the context of the updates follows from the invariant $0 \leq lo \leq hi \leq n$, the thing to be established; the summary lemma of this section bypasses the invariant by stating the result directly, and founders on tying the exit to the traces soundly. What is missing is in the engine: a rule should not be applied while the instances of its own side conditions are simplified, and a left-hand side matched through a trace variable should bind all its variables; with either, the sound form of the lemma closes the goal.